Microservice Architecture (/index.html)

Supported by Kong (https://konghq.com/)

Pattern: Access token

pattern (/tags/pattern) security (/tags/security) service design (/tags/service design)

Context

You have applied the Microservice architecture (../microservices.html) and API Gateway (../apigateway.html) patterns. The application consists of numerous services. The API gateway is the single entry point for client requests. It authenticates requests, and forwards them to other services, which might in turn invoke other services.

Problem

How to communicate the identity of the requestor to the services that handle the request?

Forces

- Services often need to verify that a user is authorized to perform an operation

Solution

The API Gateway (../apigateway.html) authenticates the request and passes an access token (e.g. JSON Web Token (https://jwt.io/)) that securely identifies the requestor in each request to the services. A service can include the access token in requests it makes to other services.

Example

See JSON Web Token (https://jwt.io/) for usage examples and supporting libraries.

Resulting context

This pattern has the following benefits:

- The identity of the requestor is securely passed around the system
- Services can verify that the requestor is authorized to perform an operation

Related patterns

- The API Gateway (../apigateway.html) uses this pattern.

---

pattern (/tags/pattern) security (/tags/security) service design (/tags/service design)

Modernization Help (https://chrisrichardson.net/)

Copyright © 2026 Chris Richardson • All rights reserved • Supported by Kong (https://konghq.com/).

ALSO ON **MICROSERVICES**

The pattern format: an antidote to hype

2 years ago · 1 comment

The pattern format: an antidote to hype Public workshops: Sept 4-6th & …

STOP hurting yourself by doing big bang …

2 years ago · 3 comments

STOP hurting yourself by doing big bang modernizations! New …

How modular can your monolith go? Part 4 - …

2 years ago · 2 comments

How modular can your monolith go? Part 4 - physical design principles …

DDD, insuf

3 years

DDD, r insuffi princip

7 Comments Login ▾

G Join the discussion...

LOG IN WITH OR SIGN UP WITH DISQUS ?

Name

♡ Share Best Newest Oldest

V **vedath**
4 years ago

We use a similar pattern, but there are problems caused by token expiry during a long-running process (think hours or days). Is there a way to extend this pattern to cover those cases?

2 0 Reply

M **machhindra nale**
7 years ago edited

We have 30 microservices being deployed as docker containers for each on-prem customer.

We don't have APIS for customers. Customer uses the exposed UIs.

1) Do we need to secure the intermicroservices communication?
2) What if we use Openshift/Kubernetes to secure the external facing endpoints and leave the inter-microservices communication in non-TLS mode?

1 0 Reply

 **omeras**
7 years ago

Great article! Do you have an article regarding authorization and authentication, which deals with how to get the access token in the first place?

1 0 Reply

 **Chris Richardson** Mod → omeras
7 years ago

See chapter 11 of my book - https://www.manning.com/boo...

3 0 Reply

S **Stefano** → Chris Richardson
6 years ago

Hi Chris, I read the chapter... I do feel like having ACLs centralized cloud the concept of boundaries:

Example: we have orderService and invoiceService, and Bob can only see 10 customers (out of 100)... therefore he can see only the orders and invoices of these 10 customers.
Options:
1. clientAuthenticationService is the only implementation of the security and gives me the dictionary of Bob his 10 customers. This service becomes a single point of failure and every other service needs to know how to apply security to their objects (orders, invoices, etc)
2. fully distribute to all services the "Bob+10 customers" (data replication) and allow service to implement the security, so we'll have order using a DB join, while invoices using a dictionary filter on their storage
3. I can use service 1 and implement the security as part of a microservices chassis maybe?
4. Include the info "Bob+10 customers" in the JWT (or Auth2) and pass it around...

Any idea? And, very important, what happens when the data becomes "Bob+10000 customers"?

0 0 Reply



Миша Путилов

2 years ago edited

There is not a word about coarse-grained authorization with access-token in the article. JWT is not a suitable medium for transferring fine-grained authorization information. It only has scopes (and roles, if you need them). Every source that I find on the internet lacks this information (sorry for being poor googler, please direct me to good sources if you find them). You can't and you should not build authorization based on scheme "subject_id permission resource_id", that's not how access_token are designed. I found just a little information about this problem in Sam Newman's book about microservices.

This website lacks a lot about authorization.

Remark: there is OAuth RAR, which supposedly solves this problem, but it is not supported by Spring Security and by KeyCloak. And I haven't investigated it thoroughly, can't say if it really solves the problem in microservices.